INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN NORMANDIE

Département STPI

Rapport de projet d'infos

INTRODUCTION AU TRAITEMENT D'IMAGES EXTRATION DE CONTOURS

Étudiants

Clément	LEFEVRE	clement.lefevre@insa-rouen.fr
Maxime	MORVANNOU	maxime.morvannou@insa-rouen.fr
Hugo	BARROIS	hugo.barrois@insa-rouen.fr
Morgan	GOUZOU–HELLOT	morgan.gouzou–hellot@insa-rouen.fr
Maïssa	BEN SETHOUM	maissa.bensethoum@insa-rouen.fr

22 septembre 2026

Encadré par

Christian GOUT

Synthèse du projet

Titre du projet : Introduction au traitement d'images – Extraction de contours

Groupe : G

Résumé : Le traitement numérique des images est un domaine clé, essentiel dans des secteurs tels que la reconnaissance de formes et l'imagerie médicale. Ce projet explore les concepts fondamentaux et les applications pratiques de ce champ, en mettant l'accent sur l'extraction de contours, une étape indispensable pour détecter les limites des objets dans une image.

Dans une première partie, les bases du traitement d'images numériques sont abordées, incluant la définition d'une image, ses caractéristiques principales (résolution, stockage, quantification), ainsi que les techniques d'amélioration de la qualité de l'image, telles que la réduction du bruit et l'amélioration du contraste. Les spécificités des images en couleur sont également traitées à travers les formats vectoriel et matriciel. Le format vectoriel décrit les formes par des équations mathématiques, garantissant une qualité optimale et une taille réduite pour des éléments simples, comme des logos. En revanche, le format matriciel, adapté aux photographies, représente une image sous forme de matrice de pixels RGB, chaque pixel étant défini par trois composantes variant entre 0 et 255.

Dans une seconde partie, le projet se concentre sur la segmentation d'images, notamment l'extraction de contours. Des méthodes variées sont étudiées, telles que le filtre de Sobel, qui détecte les variations d'intensité à l'aide de dérivées premières, et le filtre de Canny, qui combine un flou gaussien, un calcul de gradient et des seuils pour une détection précise et robuste des contours. Ces techniques reposent sur la convolution, une opération essentielle du traitement d'images. La convolution consiste à appliquer une matrice appelée "noyau" (ou kernel) à l'image. Ce noyau est une petite matrice qui, en pondérant les valeurs des pixels voisins, met en évidence des caractéristiques spécifiques, telles que les bords. Par exemple, les noyaux de Sobel sont conçus pour détecter les variations horizontales et verticales. L'opération de convolution produit une nouvelle matrice d'intensité qui accentue les zones d'intérêt (comme les contours) en calculant une somme pondérée des pixels voisins pour chaque pixel de l'image.

Ces concepts sont validés par des implémentations en Python et GLSL, permettant de tester et de comparer leurs performances, ainsi que d'illustrer leur utilité dans des applications réelles.

Mots clés : Traitement d'images numériques ; extraction de contours ; segmentation d'images ; format matriciel ; format vectoriel ; filtre de Sobel ; filtre de Prewitt ; filtre de Canny ; Laplacien ; convolution ; noyau (kernel) ; flou Gaussien ; Python ; GLSL

Table des matières

Synthèse du projet **1**

Introduction **3**

1 Organisation du travail **4**

2 Traitement numérique d'images [5][9] **5**
2.1 Image Numérique 5
2.1.1 Image vectorielle 5
2.1.2 Image matricielle 5
2.1.3 Affichage d'une image en python 6
2.1.4 Stockage et Résolution d'une Image 6
2.1.5 Quantification d'une Image 7
2.2 Traitement d'une image 7
2.2.1 Atténuation du Bruit (cf. Section 2.4.2) 7
2.2.2 Images Couleurs 8
2.2.3 Amélioration du Contraste 9
2.3 L'opération de convolution 9
2.3.1 Définition 9
2.3.2 Implémentation 10
2.3.3 Cas des bords 10
2.4 Définitions nécessaires au traitement d'images 12
2.4.1 Le flou 12
2.4.2 Le bruit d'une image 12

3 Segmentation d'images **13**
3.1 Filtre de Sobel : 13
3.1.1 Définition 13
3.1.2 Implémentation 14
3.1.3 Test 14
3.2 Filtre de Prewitt : 14
3.2.1 Définition 14
3.2.2 Implémentation 15
3.3 Différence de Gaussiennes (DoG) : 15
3.3.1 Définition 15
3.3.2 Exemples de kernels calculés grâce à $G(x, y)$ 15
3.4 Laplacien d'une Gausienne (LoG) : 16
3.4.1 Définition 16
3.4.2 Gestion du bruit 17
3.4.3 Application 17
3.5 Filtre de Canny[10][6] 17
3.5.1 Fonctionnement 17
3.5.2 Exemples 18
3.5.3 Utilité 19
3.5.4 Conclusion 19

Conclusion **20**

Introduction

Le traitement numérique des images est une discipline essentielle dans de nombreux domaines scientifiques et technologiques, allant de la reconnaissance de formes à l'imagerie médicale.

Ce projet a pour objectif d'explorer les bases théoriques et pratiques de ce domaine, avec un focus particulier sur l'extraction de contours, une technique essentielle pour identifier les limites des éléments d'une image.

Dans un premier temps, nous nous intéresserons aux fondamentaux du traitement d'images numériques. Cela inclut la définition d'une image et de ses caractéristiques essentielles, comme la résolution, le stockage et la quantification.

Nous étudierons également des techniques d'amélioration de la qualité des images, telles que la réduction du bruit et l'amélioration du contraste, ainsi que les spécificités des images en couleur.

Dans un second temps, nous approfondirons la thématique de la segmentation d'images, en mettant l'accent sur les méthodes d'extraction de contours.

Nous analyserons différentes approches, allant des techniques classiques de filtrage (par exemple, le filtre de Sobel) aux modèles plus avancés comme le filtre de Canny.

Ces filtres seront illustrés par des implémentations en Python, puis validés et commentés à travers des tests numériques.

Ce projet s'appuie sur des ressources pédagogiques solides, référencées dans la bibliographie, afin de fournir une compréhension complète des techniques de traitement et de segmentation d'images.

1 Organisation du travail

Pour ce projet, nous nous sommes répartis en plusieurs sous-groupes.

Premièrement, Hugo, ayant des compétences en langage Python, s'est occupé d'expérimenter les différentes techniques de filtrage d'images et d'extraction de contours, avec pour objectif d'en tester les limites : application aux images en couleurs, flou gaussien et kernels exotiques.

Ensuite, Morgan, grâce à son expérience passée sur Blender, maîtrisait déjà une grande partie des notions étudiées dans le cadre du traitement d'images. Grâce à cela, il a pu guider l'ensemble du groupe dans les recherches à mener.

Maïssa, avec l'aide de Morgan, a effectué des recherches sur les différentes définitions liées à l'image numérique et à son traitement, présentées dans la première partie. De plus, ils se sont chargés de l'implémentation des filtres de Sobel, de Prewitt et de Canny, en se concentrant principalement sur ce dernier.

Maxime et Clément, pour leur part, ont mené des recherches sur les Différences de Gaussiennes (DoG) ainsi que sur le Laplacien de Gaussiennes (LoG). Pour cela, ils se sont renseignés sur les notions mathématiques nécessaires à la compréhension de ces techniques. De plus, ils ont appris le codage LaTeX afin de rédiger le rapport de projet.

2 Traitement numérique d'images [5][9]

2.1 Image Numérique

Qu'est-ce qu'une image numérique ?
En informatique, il existe deux grandes catégories d'images : vectorielles et matricielles.

2.1.1 Image vectorielle

Premièrement, le format vectoriel ne stocke pas les informations pixel par pixel, mais utilise des définitions mathématiques pour décrire les formes qui composent l'image. Par exemple, un cercle sera défini par sa position centrale, son rayon et sa couleur. Cela permet de réduire la taille du fichier tout en maintenant une qualité d'image optimale, quelle que soit sa résolution. Ce format est très utilisé sur Internet pour afficher des symboles ou des logos simples. Son principal inconvénient est qu'il est moins adapté pour représenter des images complexes, comme des photographies ou des textures naturelles.

2.1.2 Image matricielle

Le format matriciel, appelé Bitmap, que nous utiliserons dans ce projet, représente une image sous la forme d'une matrice de petits éléments de couleur, appelés pixels. Chaque pixel est défini par trois variables allant de 0 à 255. Ce sont des entiers non signés, car ils sont toujours positifs et encodés sur 8 bits (ce qui permet d'exprimer des valeurs comprises entre 0 et 255 inclus). Ces trois variables représentent respectivement le rouge, le vert et le bleu. Grâce à la combinaison de ces trois composantes, il est possible de former jusqu'à 16581375 couleurs différentes. Par exemple, la couleur rouge est représentée par : $(255, 0, 0)$, le vert par $(0, 255, 0)$ et le bleu par $(0, 0, 255)$. Ce format est couramment utilisé dans des fichiers tels que .JPG, .PNG et .JPEG, parmi les plus populaires.

Ces images sont généralement stockées sous forme de tableaux de tableaux, où chaque élément du tableau est un tripler d'entiers représentant les composantes RGB d'un pixel.

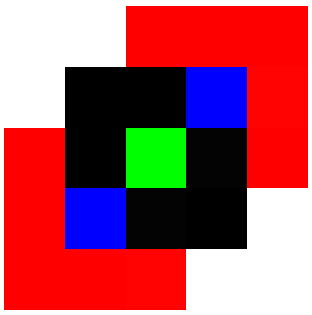


FIGURE 1 – Image Bitmap de 5 pixels par 5 pixels

La matrice équivalente :

$$\begin{bmatrix} (255,255,255) & (255,255,255) & (255,0,0) & (255,0,0) & (255,0,0) \\ (255,255,255) & (0,0,0) & (0,0,0) & (0,0,255) & (255,0,0) \\ (255,0,0) & (0,0,0) & (0,255,0) & (0,0,0) & (255,0,0) \\ (255,0,0) & (0,0,255) & (0,0,0) & (0,0,0) & (255,255,255) \\ (255,0,0) & (255,0,0) & (255,0,0) & (255,255,255) & (255,255,255) \end{bmatrix}$$

2.1.3 Affichage d'une image en python

Une image matricielle est donc une représentation numérique d'une scène ou d'un objet, constituée de pixels, chacun caractérisé par une intensité (en niveaux de gris) ou des valeurs de couleurs.Nous avons sélectionné une image sur laquelle nous avons appliqué les différents programmes. Ici le programme en Python permet de charger une image en niveaux de gris et de l'afficher.

```
import cv2
import matplotlib.pyplot as plt

# Charger une image en niveaux de gris
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Afficher l'image
plt.imshow(image, cmap='gray')
plt.title("Image en Niveaux de Gris")
plt.axis('off')"
plt.show()
```



FIGURE 2 – Affichage d'une image en niveaux de gris

2.1.4 Stockage et Résolution d'une Image

Les images numériques sont stockées sous forme de matrices de données dans différents formats (JPEG, PNG, etc.), avec une attention particulière portée à l'espace mémoire utilisé. La résolution de l'image correspond à sa taille en pixels (hauteur × largeur). Ce paramètre influence directement la qualité de l'image et la capacité à l'analyser. Ce programme permet d'afficher la résolution et la taille en mémoire de l'image.

```
# -- suite --
# Afficher les dimensions et la taille d'une image
print(f"Dimensions de l'image : {image.shape}")
print(f"Resolution (hauteur x largeur) : {image.shape[0]} x {image.shape[1]}")
print(f"Taille en memoire : {image.size * image.itemsize} octets")
```

2.1.5 Quantification d'une Image

La quantification désigne la profondeur des pixels, c'est-à-dire le nombre de bits alloués à la couleur de chaque pixel. Cela détermine le nombre de niveaux d'intensité ou de couleurs disponibles. Ce programme permet de réduire le nombre de couleurs possibles pour l'image (réduction de profondeur). Dans cet exemple, cette réduction limite l'image à seulement 4 couleurs.

```
# -- suite --
import numpy as np

# Reduire la profondeur de bits d'une image (quantification)
quantified_image = (image // 64) * 64 # Reduction a 4 niveaux d'intensite

# Afficher l'image quantifiee
plt.imshow(quantified_image, cmap='gray')
plt.title("Image Quantifiee")
plt.axis('off')
plt.show()
```

2.2 Traitement d'une image

2.2.1 Atténuation du Bruit (cf. Section 2.4.2)

En pratique, les images capturées peuvent être perturbées par du bruit. L'atténuation du bruit consiste à appliquer des filtres pour améliorer la qualité de l'image. Ici, un flou gaussien est appliqué à une image altérée par un bruit aléatoire.

```
# -- suite --
# Ajouter un bruit gaussien
noisy_image = image + np.random.normal(0, 25, image.shape).astype(np.uint8)

# Appliquer un filtre gaussien pour reduire le bruit
denoised_image = cv2.GaussianBlur(noisy_image, (10, 10), 0)

# Affichage
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(noisy_image, cmap='gray')
plt.title("Image Bruitee")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(denoised_image, cmap='gray')
plt.title("Image Debruitee")
plt.axis('off')

plt.show()
```



FIGURE 3 – Atténuation du Bruit avec un flou Gaussien de 10 par 10 pixels

2.2.2 Images Couleurs

Une image couleur est composée de plusieurs canaux (comme le rouge, le vert et le bleu – RGB), offrant une représentation riche et fidèle des scènes. Ce code permet de séparer les canaux de couleur en trois images distinctes.

```
# -- suite --
# Charger une image en couleurs
color_image = cv2.imread('image.jpg')
color_image_rgb = cv2.cvtColor(color_image, cv2.COLOR_BGR2RGB) # Conversion BGR
    -> RGB

# Afficher les canaux R, G, B
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
colors = ['Rouge', 'Vert', 'Bleu']
for i, ax in enumerate(axes):
    channel = color_image_rgb[:, :, i]
    ax.imshow(channel, cmap='gray')
    ax.set_title(f"Canal {colors[i]}")
    ax.axis('off')
plt.show()
```



FIGURE 4 – Séparation des couleurs rouge, vert et bleu

2.2.3 Amélioration du Contraste

Cette technique permet de rehausser les détails et de rendre les caractéristiques d'une image plus visibles. Cet effet peut être utilisé en appliquant cette formule :

$$\text{ImageFinal} = (\text{ImageOriginal} - 0,5) \times \alpha + 0,5$$

avec des couleurs allant de $0,0$ a $1,0$ et α qui correspond à l'intensité du contraste.

```
# -- suite --
# Appliquer une egalisation d'histogramme
equalized_image = cv2.equalizeHist(image)

# Afficher l'image originale et amelioree
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title("Image Originale")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(equalized_image, cmap='gray')
plt.title("Image avec Contraste Ameliore")
plt.axis('off')

plt.show()
```



FIGURE 5 – Amélioration du contraste

2.3 L'opération de convolution

2.3.1 Définition

La convolution consiste en une opération de multiplication entre deux matrices de tailles différentes (généralement une petite et une grande), mais de même dimensionnalité (par exemple, 1D ou 2D), produisant une nouvelle matrice de même dimensionnalité. La convolution correspond ainsi au traitement d'une matrice (par exemple, une image) par une autre petite matrice appelée matrice de convolution ou noyau (kernel).[4]

La matrice de gauche représente le kernel, qui sert à pondérer les pixels voisins. La matrice de droite représente l'image.

Cette opération aboutit à un scalaire. La formule de la convolution s'écrit ainsi :

$$\left(\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \right) [2,2] = (i \cdot 1) + (h \cdot 2) + (g \cdot 3) + (f \cdot 4) + (e \cdot 5) + (d \cdot 6) + (c \cdot 7) + (b \cdot 8) + (a \cdot 9)$$

2.3.2 Implémentation

Ce programme Python est une implémentation naïve de l'opération de convolution sur une image.

```python
def appliquer_kernel(image, kernel):
    """
    Applique un kernel (matrice) a une image representee sous forme de liste 2D.

    :param image: Liste 2D representant les pixels de l'image (grayscale).
    :param kernel: Liste 2D representant le noyau (kernel).
    :return: Nouvelle image apres convolution.
    """
    hauteur_image = len(image)
    largeur_image = len(image[0])
    hauteur_kernel = len(kernel)
    largeur_kernel = len(kernel[0])

    # Calcul des decalages pour centrer le kernel
    offset_h = hauteur_kernel // 2
    offset_w = largeur_kernel // 2

    # Image resultat
    resultat = [[0 for _ in range(largeur_image)] for _ in range(hauteur_image)]

    # Convolution
    for i in range(offset_h, hauteur_image - offset_h):
        for j in range(offset_w, largeur_image - offset_w):
            somme = 0
            for ki in range(hauteur_kernel):
                for kj in range(largeur_kernel):
                    pixel = image[i + ki - offset_h][j + kj - offset_w]
                    coeff = kernel[ki][kj]
                    somme += pixel * coeff
            resultat[i][j] = somme

    return resultat
```

2.3.3 Cas des bords

Cette opération est réalisée sur tous les pixels de l'image, ce qui nous amène à un premier problème : dans le cas des pixels se trouvant sur les bords de l'image, l'opération de convolution va essayer d'accéder à des pixels hors de l'image. Pour remédier à ce problème, on peut étendre les bords de différentes façons.

1. **Extension :** Le pixel au bord est répété autant que nécessaire en ligne.
2. **Wrap :** L'image est répétée en reprenant les pixels du côté opposé.
3. **Mirror :** L'image est répétée en retournant l'image et en reprenant les pixels du même côté.

On peut aussi ne pas considérer les pixels des bords, mais cela donnerait une image plus petite après traitement.



FIGURE 6 – Exemple continuité des bords

Sur cette figure, nous avons sélectionné une image sur laquelle nous avons appliqué les différentes méthodes d'extension des bords. En pratique, l'extension est de la taille du noyau, ici les effets sur les images sont accentués pour que l'effet soit plus visible. L'effet est déterminé en fonction du contenu de l'image, et on compare les différentes méthodes pour voir laquelle est la plus intéressante. Ici, la méthode du miroir semble la plus adaptée.

2.4 Définitions nécessaires au traitement d'images

2.4.1 Le flou

Le flou dans une image est une réduction ou une perte de netteté qui entraîne une atténuation des détails et des contours. Cela se manifeste par une transition plus douce entre les pixels, rendant les limites moins distinctes. Le flou est souvent utilisé pour adoucir l'image et réduire les variations mineures (comme le bruit) avant de détecter les contours. En lissant l'image, on facilite la détection des contours significatifs en éliminant les détails superflus et les variations rapides dues à des imperfections ou des petites textures. Par exemple, des techniques comme le flou Gaussien sont couramment employées pour cet objectif. [8]

2.4.2 Le bruit d'une image

Le bruit dans une image est une perturbation aléatoire qui dégrade sa qualité en y introduisant des variations de couleur ou de luminosité qui ne correspondent pas aux détails réels de l'image. Il peut provenir du capteur de la caméra (surtout en faible lumière), d'erreurs lors de la compression ou de conditions de capture difficiles. On retrouve trois grands types de bruit : le bruit gaussien, qui correspond à de petites variations floues, le bruit "poivre et sel", qui se traduit par des points noirs et blancs dispersés, et le bruit "speckle", qui peut être défini par de la granularité spécifique à certaines images (comme les échographies). Le bruit perturbe les algorithmes d'analyse, notamment l'extraction de contours, car il crée de faux détails. Il est souvent atténué par des filtres (comme le flou gaussien ou des techniques de lissage) avant d'appliquer des étapes de traitement plus avancées. [7] [2] [3]

3 Segmentation d'images

L'extraction de contours est une étape clé en traitement d'images, utilisée pour détecter les frontières des objets en analysant les variations d'intensité. Cette section explore plusieurs méthodes, allant des techniques classiques de filtrage, comme le filtre de Sobel, aux techniques plus avancées telles que le filtre de Canny. Des implémentations en Python mais également en GLSL (pour de meilleures performances), notamment pour le filtre de Sobel, permettront de valider ces concepts et de comparer leurs performances. Nous allons, à partir de maintenant, utiliser des images matricielles en niveaux de gris.

3.1 Filtre de Sobel :

Dans le cas où nous utilisons un seul noyau pour la détection de contours, nous n'avons pas assez d'informations pour déterminer l'angle de celui-ci, nous pouvons seulement extraire l'intensité de ce contour.

Voici des exemples de noyaux pour un filtre de détection de contours :

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} \text{ ou } \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

3.1.1 Définition

Le filtre de Sobel est une méthode de détection de contours qui repose sur les dérivées premières d'une image pour identifier les variations rapides d'intensité, typiques des bords.

Il utilise deux masques de convolution (noyaux), l'un pour détecter les variations horizontales (en X) et l'autre pour les variations verticales (en Y).

Noyau du filtre de Sobel :

En X :

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

En Y :

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

On peut filtrer en fonction de la norme du gradient :

$$\|\vec{\mathbf{grad}}\| = \sqrt{\mathbf{grad}_x^2 + \mathbf{grad}_y^2}$$

On peut obtenir l'angle du contour :

$$\theta = \arctan\left(\frac{\mathbf{grad}_y}{\mathbf{grad}_x}\right)$$

3.1.2 Implémentation

```
kernel_sobel_X = [[-1, 0, 1],
                  [-2, 0, 2],
                  [-1, 0, 1]]
kernel_sobel_Y = [[-1,-2,-1],
                  [ 0, 0, 0],
                  [ 1, 2, 1]]
image_gradient_X = appliquer_kernel(image, kernel_sobel_X)
image_gradient_Y = appliquer_kernel(image, kernel_sobel_Y)
```

Ces quelques lignes de programme montrent comment appliquer le noyau de Sobel dans un programme en utilisant la fonction définie précédemment.

3.1.3 Test



FIGURE 7 – Comparaison Filtre de Sobel

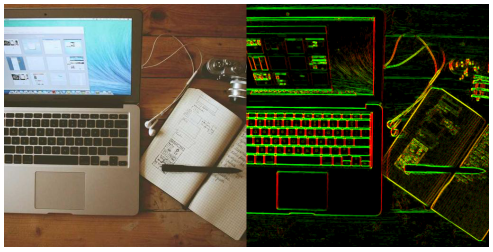


FIGURE 8 – Comparaison Filtre de Sobel

Dans ces exemples, nous avons appliqué notre code à deux images, ce qui nous a permis d'obtenir les deux résultats présentés ci-dessus. On remarque que les contours ont été correctement extraits, avec les pixels en rouges représentant les variations en X et les pixels en verts représentant les variations en Y.

3.2 Filtre de Prewitt :

3.2.1 Définition

De manière similaire au filtre de Sobel, on applique le filtre de Prewitt en X sur l'image originale, puis on fait de même pour le filtre en Y, ce qui nous donne les dérivées de l'image selon les axes X et Y, permettant d'obtenir le gradient.

kernel filtre de prewitt :

En X :

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

En Y :

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

3.2.2 Implémentation

```
kernel_prewitt_X = [[-1, 0, 1],
                    [-1, 0, 1],
                    [-1, 0, 1]]
kernel_prewitt_Y = [[-1,-1,-1],
                    [ 0, 0, 0],
                    [ 1, 1, 1]]
image_gradient_X = appliquer_kernel(image, kernel_prewitt_X)
image_gradient_Y = appliquer_kernel(image, kernel_prewitt_Y)
```

Ces lignes de code montrent comment appliquer le noyau de Prewitt dans le programme de convolution. (cf. section 2.3) Après plusieurs tests sur différentes images, nous avons remarqué que les résultats étaient semblables à ceux obtenus avec le filtre de Sobel.

Le filtre de Sobel et le filtre de Prewitt sont tous deux utilisés pour détecter les contours dans une image en mesurant les changements de luminosité. Le filtre de Sobel est plus précis et mieux adapté aux images bruyantes, tandis que le filtre de Prewitt est plus simple et rapide, mais moins précis. Les deux filtres sont efficaces pour des tâches comme la détection de bords, mais le choix entre les deux dépend des besoins spécifiques, comme la rapidité ou la précision.

3.3 Différence de Gaussiennes (DoG) :

3.3.1 Définition

La fonction gaussienne est une fonction mathématique utilisée pour lisser une image. Voici son expression en 2 dimensions (dans le cas d'une image) :

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

— $G(x,y)$ est la fonction gaussienne à 2 dimensions.
— x et y représentent les coordonnées spatiales des pixels dans l'image.
— σ est l'écart-type de la Gaussienne, qui contrôle le niveau de lissage (plus σ est grand, plus les pixels voisins ont du poids dans la moyenne).

Cette fonction gaussienne est appliquée à chaque pixel de l'image, permettant de réduire les petites variations d'intensité et donc le bruit.

Pour appliquer ce filtre, on utilise un noyau obtenu en appliquant la fonction $G(x,y)$ sur chaque élément de la matrice, en prenant son centre comme origine pour x et y. On normalise ensuite la matrice pour que la somme des éléments donne 1.

La différence de gaussiennes se fait en soustrayant deux images avec des intensités de flou différentes. Cette opération permet d'extraire les contours d'une image ainsi que son bruit. L'inconvénient de cette méthode est qu'elle ne permet pas d'extraire d'informations concernant l'angle du contour, comme pour la détection de contours à un seul noyau. Malgré cela, ce filtre reste intéressant car il est beaucoup plus lisse que les autres méthodes.

3.3.2 Exemples de kernels calculés grâce à $G(x,y)$

kernel flou gaussien (3x3) :

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

kernel flou gaussien (5x5) :

$$\frac{1}{256}\begin{bmatrix}1 & 4 & 6 & 4 & 1\\ 4 & 16 & 24 & 16 & 4\\ 6 & 24 & 36 & 24 & 6\\ 4 & 16 & 24 & 16 & 4\\ 1 & 4 & 6 & 4 & 1\end{bmatrix}$$

3.4 Laplacien d'une Gausienne (LoG) :

3.4.1 Définition

Le Laplacien est un opérateur différentiel qui mesure la somme des dérivées secondes d'une fonction (dans ce cas, l'intensité de l'image en noir et blanc). En deux dimensions, le Laplacien d'une fonction $f(x, y)$ (qui peut être l'image ou une image floutée par une gaussienne) s'exprime ainsi :

$$\nabla^2 f(x,y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

C'est la somme des dérivées secondes par rapport à x et à y. En termes simples, cela signifie que le Laplacien mesure la courbure locale de l'intensité lumineuse dans l'image. Là où cette courbure change rapidement (par exemple, aux bords ou contours d'objets), le Laplacien sera très grand. On peut donc localiser les contours en observant où le Laplacien s'annule, ce qui correspond à un point d'inflexion.

L'opérateur s'applique sur des fonctions continues. Or, notre image est composée de pixels. On doit donc passer par une approximation discrète de l'opérateur. Voici quelques approximations :

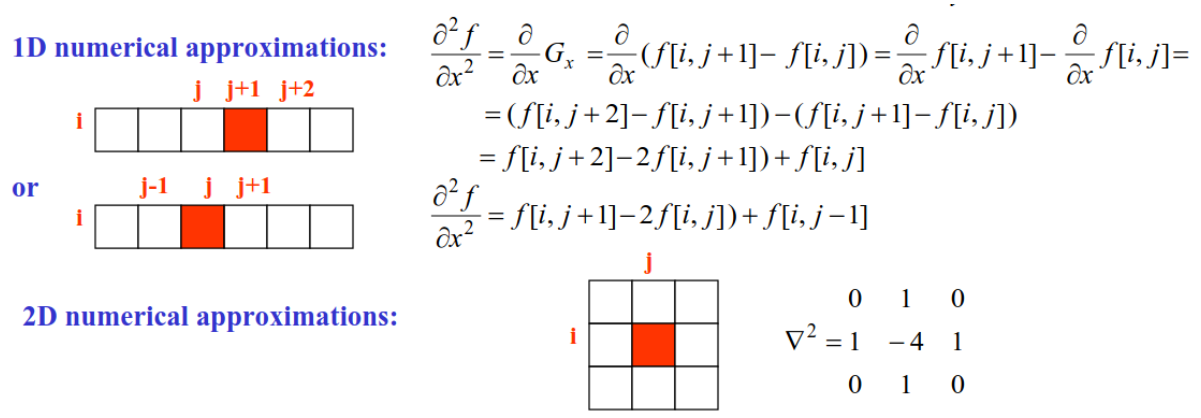


FIGURE 9 – Approximation du Laplacien en 1D et 2 D

3.4.2 Gestion du bruit

Les opérateurs dérivés présentés jusqu'à présent ne sont pas très utiles car ils sont très sensibles au bruit. Pour atténuer le bruit avant le filtrage, Marr et Hildreth ont proposé un filtre gaussien (voir DoG), combiné avec le Laplacien pour la détection des contours. Cela donne le Laplacien d'une Gaussienne (LoG).

Le Laplacien d'une Gaussienne (LoG) combine ces deux étapes en un seul calcul. On applique le Laplacien directement à la fonction gaussienne $G(x, y)$, ce qui donne :

$$LoG(x, y) = \nabla^2 G(x, y)$$

3.4.3 Application

Le résultat du calcul est un filtre qui peut être appliqué à une image pour détecter ses contours. En calculant cette opération, on trouve la dérivée seconde de la fonction gaussienne :

$$\nabla^2 G(x, y) = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Ce noyau (ou masque de convolution) est ensuite utilisé pour filtrer l'image en la convoluant avec chaque pixel. C'est une opération qui calcule une somme pondérée des pixels environnants à chaque point et met en évidence les zones où il y a des changements brusques d'intensité, correspondant aux contours.
Voici les étapes du filtre :

- Smoothing (Lissage) : Appliquer la fonction gaussienne à chaque pixel de l'image pour la lisser. Cela réduit le bruit et les petites variations.
- Laplacian (Dérivation) : Appliquer l'opérateur laplacien sur l'image lissée pour détecter les zones de changement rapide d'intensité, c'est-à-dire les contours.
- Seuil : Enfin, on peut appliquer un seuil pour ne garder que les valeurs les plus importantes du résultat, celles qui correspondent aux contours les plus nets.

En pratique, tout cela est souvent fait en une seule étape en appliquant le noyau pré-calculé du Laplacien d'une Gaussienne à l'image via une convolution.[1]

3.5 Filtre de Canny[10][6]

3.5.1 Fonctionnement

Le filtre de Canny est un algorithme de détection de contours possédant une grande précision, développé par John F. Canny en 1986. Il est considéré comme l'un des meilleurs algorithmes de détection de contours grâce à son habileté à minimiser le bruit et à détecter précisément les vrais contours tout en ignorant les faux. Cet algorithme peut être décrit en plusieurs étapes :

1. Atténuation du bruit avec un flou Gaussien : La détection de contours est sensible au bruit, on applique alors un flou Gaussien pour réduire les anomalies. Les petites variations sont atténuées, ne laissant que les contours importants.

2. Calcul du gradient de l'image : On applique le filtre de Sobel mentionné précédemment sur l'image. Cette opération permet d'obtenir l'intensité et l'angle du

gradient.

3. Suppression des Non-Maximas : La suppression des Non-maximas permet d'affiner les contours extraits avec le filtre de Sobel. Pour déterminer le maximum local, on compare le pixel actuel avec ses voisins dans la direction du gradient. Les pixels qui ne sont pas des maxima sont alors supprimés. Cela permet de ne garder que les contours significatifs et assure, dans la plupart des cas, que les contours ont une épaisseur d'un seul pixel.

4. Double seuil : Le filtre de Canny utilise deux seuils, un haut et un bas, pour identifier les contours forts et faibles.
   — Les contours forts sont les pixels avec un gradient plus élevé que le seuil haut.
   — Les contours faibles sont les pixels avec un gradient entre le seuil haut et le seuil bas et ne sont gardés que s'ils sont connectés à un contour fort.
   — Les pixels ayant un gradient inférieur au seuil bas ne sont pas conservés.
   Cette étape permet d'enlever les contours faibles dus au bruit tout en gardant les contours significatifs connectés aux contours forts.

3.5.2 Exemples

Nous avons sélectionné une image sur laquelle nous avons appliqué un programme implémenté en GLSL. Afin de mieux visualiser l'influence des seuils, nous les avons fait varier. En théorie, le filtre de Canny utilise deux seuils, un maximum et un minimum. Cependant, lors de la pratique, nous avons implémenté un programme utilisant un seuil pour le filtre de Canny et un autre seuil pour le filtre de Sobel. En ordonnée, nous avons le seuil du filtre de Canny et en abscisse, le seuil du filtre de Sobel.

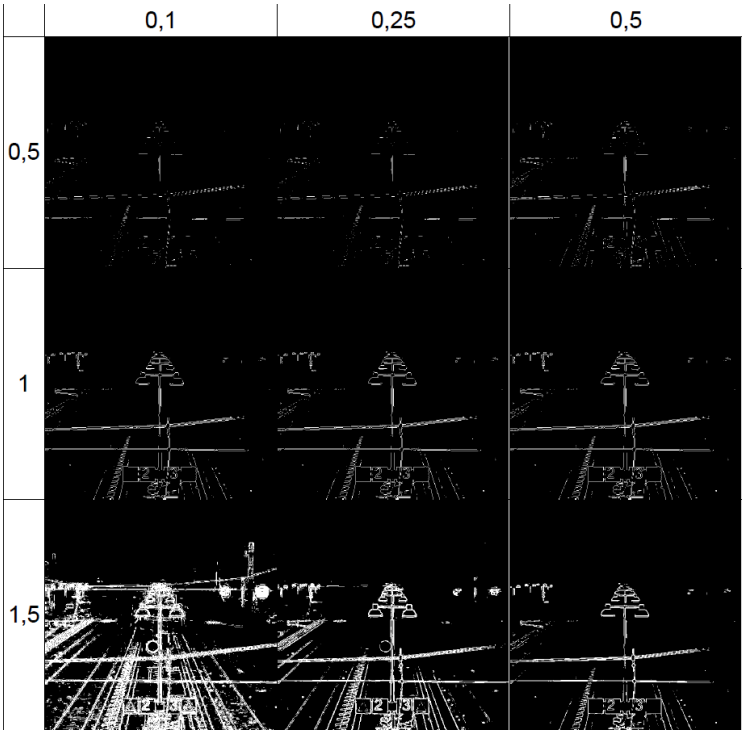


FIGURE 10 – Tableau d'image des résultats en fonction des différents seuils

On remarque que l'image avec un seuil élevé pour les deux filtres donne une segmentation de bonne qualité. Cependant, pour l'image du centre, le résultat est satisfaisant. Cela montre que les paramètres de seuils dépendent non seulement de l'image sélectionnée, mais aussi du résultat attendu.

3.5.3 Utilité

Le filtre de Canny est essentiel pour extraire les zones d'intérêt et préparer les images à des traitements comme la segmentation ou la reconnaissance d'objets. Très utilisé en vision par ordinateur, il sert à détecter des formes, suivre des objets en vidéo ou analyser des images satellites. En imagerie médicale, il permet de délimiter des structures comme des organes. Sa précision et sa robustesse face au bruit en font un outil polyvalent pour des applications variées.

3.5.4 Conclusion

En conclusion, le filtre de Canny détecte les contours en suivant plusieurs étapes simples. Il commence par lisser l'image avec un flou gaussien pour réduire le bruit, puis calcule les gradients pour repérer les zones où l'intensité change brusquement. Ensuite, il affine ces résultats en supprimant les faux contours et applique un double seuil pour séparer les contours forts des faibles. Enfin, il connecte les contours pertinents pour produire une détection précise et propre. Cette méthode, bien structurée, explique sa fiabilité et son efficacité.

Synthèse des filtres

Voici un tableau récapitulatif comparant les filtres présentés précédemment. La méthode de Canny se distingue comme étant la plus performante pour des applications exigeant précision et robustesse, mais au prix d'une complexité plus importante. En revanche, Sobel et Prewitt conviennent pour des calculs rapides et simples, bien qu'ils soient moins précis et robustes. La méthode LoG offre un bon compromis entre performance et adaptabilité.

Comparaison synthétique

Méthode	Robustesse au bruit	Précision des contours	Complexité de calcul	Adaptabilité
Laplacien d'une Gaussienne (LoG)	Moyenne	Bonne	Moyenne	Adaptable (multi-échelle)
Sobel	Faible	Moyenne	Faible	Limitée
Prewitt	Faible	Moyenne	Très faible	Limitée
Canny	Excellente	Excellente	Élevée	Très adaptable

FIGURE 11 – Comparaison des différents filtres

Conclusion

Ce projet a permis d'explorer les bases du traitement d'images numériques, avec un focus particulier sur l'extraction de contours, une technique cruciale pour analyser les limites et les structures d'une image. Nous avons abordé des concepts fondamentaux tels que la résolution, le stockage et la quantification des images, et avons examiné des méthodes d'amélioration de la qualité des images, telles que la réduction du bruit et l'amélioration du contraste.

L'une des étapes centrales de ce projet a été l'analyse de différentes techniques d'extraction de contours, notamment les filtres de Sobel, de Canny, et le Laplacien. Chacun de ces filtres présente des avantages et des limitations qui dépendent largement des caractéristiques de l'image à traiter.

Le filtre de Sobel, relativement simple et rapide, est efficace pour détecter les bords dans des images aux contours nets, mais il reste sensible au bruit, ce qui peut nuire à sa précision dans des images plus complexes. Le filtre de Canny, plus avancé, fournit des résultats beaucoup plus précis et plus facilement exploitables, en particulier lorsqu'il s'agit de traiter des images avec du bruit. Cependant, il est plus coûteux en termes de calculs et de temps de traitement et ses paramètres dépendent grandement de l'image à traiter, ce qui peut poser un problème pour des applications en temps réel. Enfin, le Laplacien est un opérateur différentiel utile pour détecter des changements brusques d'intensité, mais il peut également être sensible au bruit, ce qui nécessite souvent de le combiner avec un flou pour de meilleurs résultats.

Dans la vie de tous les jours, ces filtres sont largement utilisés dans des domaines tels que le milieu artistique, la reconnaissance de formes, la vision par ordinateur et la surveillance vidéo. Par exemple, le filtre de Canny est couramment utilisé pour des systèmes de sécurité ou dans des technologies telles que la conduite autonome, où une détection précise des bords est nécessaire pour la reconnaissance d'objets ou la navigation.

En conclusion, bien que chaque méthode d'extraction de contours présente des avantages et des inconvénients spécifiques, elles restent toutes fondamentales dans le domaine du traitement d'images numériques. Le choix du filtre dépend des contraintes spécifiques de l'application, mais dans l'ensemble, ces outils sont essentiels pour des applications allant de l'imagerie médicale à la robotique, en passant par la reconnaissance d'images et la vision artificielle. Leur utilité et leur efficacité ne font aucun doute, car elles continuent à être des piliers du traitement numérique des images dans de nombreux domaines.

Bilans personnalisés

Clément LEFEVRE : Je ne possédais, au début, quasiment aucune connaissance dans ce domaine. Je me suis donc concentré sur quelques recherches documentaires ainsi que sur la forme de notre compte-rendu. Pour cela il m'a fallu apprendre l'utilisation de LATEX qui me sera utile lors de mes compte-rendus de projet à l'avenir, qu'ils fassent partie de mes études ou bien de mon futur métier. Ce projet m'a également permit de voir une application direct des mathématiques qui n'ont été jusqu'à présent que théoriques. Je ne pense pas continuer dans ce domaine à l'avenir mais ce projet a éveillé ma curiosité et était un bon entraînement au travail d'équipe.

Maxime MORVANNOU : Ce projet m'a permit d'apprendre sur le travail de recherche en duo avec Clément. J'ai appris durant ce projet une autre manière de se servir des matrices qu'on avait déjà vu en cours l'année dernière. Je trouve que c'est intéressant de voir une manière concrète d'utiliser ces outils mathématiques. De plus, voulant aller dans le département mathématiques appliqués, je vois ce projet comme un aperçu du travail que j'aurais à faire plus tard.

Hugo BARROIS : Je suis content d'avoir pu travailler sur ce projet de traitement d'image. Mes connaissances préalables en Python m'ont permis de plus facilement m'immerger dans le projet et donc de vite pouvoir expérimenter les limites des filtres. Avant cela je n'avais jamais entendu parler de convolution et ne savais pas comment fonctionnaient les filtres. J'ai donc mené des recherches et ai expérimenté sur l'entièreté du sujet, ce qui m'a permis de partager mes connaissances avec le groupe. Enfin, j'ai apprécié voir et manipuler une application des mathématiques ainsi que l'apprentissage et approfondissement des outils comme la convolution et la dérivation en deux dimensions.

Morgan GOUZOU–HELLOT : Ce premier projet mathématique sur l'extraction de contours a été une expérience enrichissante, tant sur le plan académique que personnel. Il m'a permis de mieux comprendre comment les mathématiques peuvent s'ancrer concrètement dans de nombreux domaines, en particulier le traitement d'images. Le format du compte rendu, qui m'était jusqu'alors inconnu, m'a donné l'opportunité de me familiariser avec le langage LaTeX pour la rédaction scientifique.

J'ai également pu mettre à profit mes connaissances en Python et en GLSL, bien que limitées, pour l'implémentation de certains filtres. Travailler en équipe s'est avéré particulièrement enrichissant, car cela nous a permis de confronter nos points de vue, de partager nos idées et d'extraire le meilleur de nos recherches et expérimentations.

Maïssa BEN SETHOUM : Tout au long de ce projet mathématique, j'ai eu l'occasion de découvrir et d'approfondir un domaine qui m'était jusque-là inconnu : le traitement d'image. J'ai d'abord trouvé très intéressant d'en apprendre davantage sur la définition d'une image, ainsi que sur tout ce qui s'y rapporte, notamment la manière dont elle est stockée, le principe du pixel et la gestion des couleurs. Par ailleurs, j'ai également pu constater l'alliance entre des principes mathématiques et des outils informatiques. Cela m'a permis de découvrir des applications concrètes de ces notions, ainsi que leur utilité dans la vie quotidienne. Enfin, cette expérience a été enrichissante, car elle m'a permis de mieux comprendre l'organisation et le déroulement d'un projet au sein d'un groupe.

Références

[1] A.Campilho. Laplacien-d'une-gausienne. https://web.fe.up.pt/~campilho/PDI/NOTES/EdgeDetection.pdf. consulté le 12/11/2024.

[2] Harry Collis. les-types-de-bruit. https://www.photohc.com/blog/2019/12/23/les-types-de-bruit. consulté le 12/11/2024.

[3] Maxime DESCOTEAUX. Image, qualité, bruit et débruitage. https://scil.usherbrooke.ca/imn_courses/imn530/cours/imagequalitedebruitage.pdf. consulté le 12/11/2024.

[4] Loris Foresti et Pascal Horton. Filtres et convolution. https://wp.unil.ch/risk/files/2015/12/8.-Filtres-et-convolution.pdf, 2015. consulté le 10/10/2024.

[5] Gabriel Peyré. Le traitement numérique des images. https://www.apmep.fr/Le-traitement-numerique-des-images-10445. consulté le 28/11/2024.

[6] Jean-Hugh THOMAS. Détection de contours. https://http://www.optique-ingenieur.org/fr/cours/pdf/OPI_fr_M04_C05.pdf. consulté le 12/11/2024.

[7] valentin. quest-ce-que-le-bruit-en-photographie. https://vivre-de-la-photo.fr/quest-ce-que-le-bruit-en-photographie/. consulté le 12/11/2024.

[8] Wikipedia. Flou-artistique. https://fr.wikipedia.org/wiki/Flou_artistique. consulté le 12/11/2024.

[9] Wikipedia. Image numérique. https://fr.wikipedia.org/wiki/Image_num%C3%A9rique. consulté le 28/11/2024.

[10] Wikipedia. Le filtre de canny. https://fr.wikipedia.org/wiki/Filtre_de_Canny#:~:text=Le%20filtre%20de%20Canny%20. consulté le 12/11/2024.